Module 02 — Variables, Datatypes, Operators, and I/O

Phase: Core Python
Previous: Python Setup and First Programs
Next: Flow Control

Module Outcome

Store values in variables, choose the right type, compute with operators, read `input()`, cast for math, and print clear f-string output.

By the end of this module you will be able to:

- Bind and reuse a labelled value (`snake_case`)
- Use `int`, `float`, `str`, `bool`, and `None` — and check with `type()`
- Compute with `/`, `//`, `%`, `**`, and update with `+=`
- Combine comparisons (`==`, `>=`) and logic (`and`, `or`, `not`) as `True` / `False` expressions
- Check `"@" in email` and `status is None`
- Convert `input()` text with `int` / `float` before doing math
- Format money and totals with f-strings (`:.2f`, `:,`)

---

1. Why This Concept Matters

A program must **remember**. Without variables you retype every number and cannot reuse a total.

Think of a labelled box:

```
student_name  →  Ravi Kumar
```

Campus marks, a canteen line, a UPI amount — each lives in a box, then you calculate and print. Python is **dynamically typed**: the type comes from the value, not from a declaration like Java's `int x = 5`.

Next module will wrap these expressions in `if` and loops. Today you store, compute, and print.

---

2. Variables

Understand

A **variable** is a name bound to a value with `=`. Using the name later reads the current value. Binding the name again replaces what it points to.

Programs store roll numbers, marks, and expenses so later lines can calculate without hard-coding every figure.

How It Works

```
roll = 2026001
name = "Priya Nair"
marks_python = 78
marks_sql = 82
average = (marks_python + marks_sql) / 2
```

```
print("Roll:", roll)
print("Name:", name)
print(f"Average marks: {average:.2f}")
```

Expected Output:

```
Roll: 2026001
Name: Priya Nair
Average marks: 80.00
```

```
food = 185.50
travel = 60.0
total = food + travel
print(f"Today's total: ₹{total:.2f}")
```

Expected Output:

```
Today's total: ₹245.50
```

⚠️ WATCH OUT

```
# Wrong — a typo is a new name
print(salery)          # NameError if you only created salary

# Prefer — same spelling every time
salary = 25000
print(salary)
```

Try It

Store Ravi's roll `"2026002"`, name `"Ravi"`, Python marks `81`, SQL marks `74`. Print `{name} ({roll}) average: {average:.2f}`.

---

3. Core datatypes

Understand

The **type** is the kind of value in the box. Wrong type → wrong math or a crash.

Type	Example	Typical use
`int`	`42`	Counts, qty
`float`	`99.99`	Money, percentages
`str`	`"EMP001"`	Names, emails, IDs with leading zeros
`bool`	`True` / `False`	Flags (`is_active`)
`None`	`None`	Missing value (no manager yet)

Check with `type(value)`. Check a missing field with `is None`.

How It Works

```
employee_id = "E1024"      # str — keep IDs as text
years_experience = 3       # int
ctc_lakh = 4.5             # float
is_remote = True           # bool
manager_id = None          # missing

print(type(employee_id))
print(type(years_experience))
print(manager_id is None)
```

Expected Output:

```
<class 'str'>
<class 'int'>
True
```

⚠️ WATCH OUT

```
# Wrong — style/identity trap
manager_id == None

# Prefer
manager_id is None

# Wrong — treating "E1024" as an int
# letters and leading zeros must stay text
```

Try It

For Priya, set `emp_id`, `years`, `ctc_lakhs`, `is_remote`, and `manager_id`. Print `type(...)` of each on one line.

---

4. Arithmetic operators

Understand

Operators compute a new number from existing ones. Marks averages, bill splits, and GST all need this.

Operator	Meaning	Example
`+` `-` `*`	Add, subtract, multiply	`17 + 5 → 22`
`/`	True division (always `float`)	`17 / 5 → 3.4`
`//`	Floor division	`17 // 5 → 3`
`%`	Remainder	`17 % 5 → 2`
`**`	Power	`2 ** 4 → 16`

How It Works

```
a = 17
b = 5
print(a + b)
print(a / b)
print(a // b)
print(a % b)
print(a ** 2)
```

Expected Output:

```
22
3.4
3
2
289
```

A bill split shows `/` vs `//` vs `%` on the same numbers:

```
bill = 999
friends = 3
print(bill / friends)
print(bill // friends)
print(bill % friends)
```

Expected Output:

```
333.0
333
0
```

Product line with 18% GST:

```
price = 499.0
qty = 2
line_total = price * qty * 1.18
print(f"Line total with GST: ₹{line_total:.2f}")
```

Expected Output:

```
Line total with GST: ₹1177.64
```

Tonight's canteen kit uses **5%** GST (`× 1.05`) — same idea, different rate.

⚠ WATCH OUT

```
# Wrong expectation — 5 / 2 is 2  (that was old Python 2)
# Prefer — 5 / 2 → 2.5; use // when you want whole units
```

For money on screen, prefer `f"{amount:.2f}"`.

Try It

Price `80`, qty `3`, add 5% GST (`* 1.05`). Print the line total with 2 decimals and ₹.

---

5. Comparison, logical, and assignment

Understand

Comparisons produce `True` or `False`. Logical operators combine them. Assignment shortcuts update a box in place.

You will soon wrap these in `if` (next module). Today you **print the expression**.

Kind	Symbols
Comparison	`==` `!=` `<` `>` `<=` `>=`
Logical	`and` (both), `or` (at least one), `not`
Shortcuts	`+=` `-=` `*=`

How It Works

```
marks = 72
print(marks >= 35)
print(marks == 100)

username_ok = True
password_ok = False
can_login = username_ok and password_ok
print(can_login)

has_coupon = False
is_member = True
print(has_coupon or is_member)
```

Expected Output:

```
True
False
False
True
```

```
balance = 8000
balance += 1500   # deposit
balance -= 2000   # withdraw
print(f"Final balance: ₹{balance}")
```

Expected Output:

```
Final balance: ₹7500
```

⚠ WATCH OUT

```
# Wrong — single = assigns; it does not compare
# Prefer — == for equality; = only to store a value
```

Try It

Start `month_total = 0`. Add `120`, `85.5`, and `40` with `+=`. Print the total with ₹.

---

6. Membership (`in`) and identity (`is`)

Understand

`in` / `not in` ask “is this piece inside that string?”
`is` / `is not` ask “is this the **same object**?” — everyday use is `is None`.

Prefer `==` for normal value comparison (`"Ravi" == name`).

How It Works

```
email = "ravi@company.com"
print("@" in email)
print("gmail" in email)

status = None
print(status is None)
```

Expected Output:

```
True
False
True
```

⚠ WATCH OUT

```
# Wrong — using is to compare ordinary strings or numbers
# Prefer — == for values; is None for missing
```

Try It

For `priya@ksr.in`, print whether it contains `@`, then whether it contains `.in`.

---

7. `input()` and type conversion

Understand

`input()` always returns a **string**. Forms and CLIs read text. Averages and GST need numbers — so you **cast**.

```
User types 250
     ↓
input() → "250"   (str)
     ↓
```

```
float("250") → 250.0
     ↓
Ready for calculation
```

Call	Use
`int("42")`	Whole numbers (qty, roll digits)
`float("12.5")`	Money, marks with a decimal
`str(42)`	Turn a number into text

Invalid cast → `ValueError` (`int("abc")`). Handling that crash comes in Exceptions week.

How It Works

Live (type when you run it — output depends on what you type):

```
print("--- Expense Tracker ---")
category = input("Category (food/travel/other): ")
amount = float(input("Amount in ₹: "))
print(f"Saved: {category} → ₹{amount:.2f}")
```

Same logic with values already in variables so you can check the output:

```
category = "food"
amount = float("250")
print(f"Saved: {category} → ₹{amount:.2f}")
```

Expected Output:

```
Saved: food → ₹250.00
```

⚠️ WATCH OUT

```
# Wrong
input("Amount: ") + 100     # str + int → TypeError
int("12.5")                 # ValueError — int does not accept a decimal point

# Prefer
float(input("Amount: ")) + 100
float("12.5")
```

Try It

Simulate `input` results: category `"travel"` and amount text `"80.5"`. Cast and print `Saved: travel → ₹80.50`.

---

8. f-strings (daily driver)

Understand

A string prefixed with `f` embeds expressions in `{ }`. Clearer than gluing pieces with `+`.

Useful bits:

- `{salary:,}` — thousands separator
- `{total:.2f}` — two decimal places

How It Works

```
name = "Karthik"
role = "Backend Intern"
salary = 25000
print(f"{name} — {role} — ₹{salary:,}/month")
```

Expected Output:

```
Karthik — Backend Intern — ₹25,000/month
```

⚠️ WATCH OUT

```
# Wrong
"Age: " + 22          # TypeError — str + int

# Prefer
f"Age: {22}"
```

Try It

Print `Ravi — Intern — ₹25,000/month` using variables and an f-string with `:,`.

Step-by-Step: Build It

One marks calculator: store, add, divide, compare, print. Hard-coded numbers so the output is fixed. In class, swap `float(input(...))` for each mark when you demo live.

```
m1 = 78
m2 = 82
m3 = 91
total = m1 + m2 + m3
average = total / 3
print(f"Total: {total}")
print(f"Average: {average:.2f}")
print("Pass:", average >= 35)
```

Expected Output:

```
Total: 251
Average: 83.67
Pass: True
```

Live variant (you type the three marks):

```
print("=== Student Marks Calculator ===")
m1 = float(input("Python marks: "))
m2 = float(input("SQL marks: "))
m3 = float(input("HTML marks: "))

total = m1 + m2 + m3
average = total / 3

print(f"Total: {total:.2f}")
print(f"Average: {average:.2f}")
print("Pass?", average >= 35)
```

Sample Run (typed `78`, `82`, `91`):

```
=== Student Marks Calculator ===
Python marks: 78
SQL marks: 82
HTML marks: 91
Total: 251.00
Average: 83.67
Pass? True
```

Tonight's kits use the same tools on a **canteen line** (price × qty + GST) and on **casting bugs** — not this marks script.

Practice Exercises

Attempt each problem before reading the model answer.

1. Hostel bill: rent `8500` and mess `3200`. Print today's total with ₹ and 2 decimals.

Model answer:

```
rent = 8500
mess = 3200
total = rent + mess
print(f"Today's total: ₹{total:.2f}")
```

Expected Output:

```
Today's total: ₹11700.00
```

2. Convert `250` minutes into hours and leftover minutes using `//` and `%`. Print both.

Hint: `60` minutes in an hour.

Model answer:

```
minutes = 250
hours = minutes // 60
left = minutes % 60
print(hours)
print(left)
```

Expected Output:

```
4
10
```

3. Attendance: `present = 42`, `sessions = 50`. Print the percentage with 2 decimals, then print whether it is `>= 75`.

Model answer:

```
present = 42
sessions = 50
percent = present / sessions * 100
print(f"{percent:.2f}")
print(percent >= 75)
```

Expected Output:

```
84.00
True
```

4. Simulate `input` results: qty text `"3"` and price text `"149.0"`. Cast, compute line total, print `Line: ₹447.00`.

Model answer:

```
qty = int("3")
price = float("149.0")
line = qty * price
print(f"Line: ₹{line:.2f}")
```

Expected Output:

```
Line: ₹447.00
```

5. Wallet starts at `1000`. Add `200` with `+=`, subtract `350` with `-=`. Print the wallet.

Model answer:

```
wallet = 1000
wallet += 200
wallet -= 350
print(f"Wallet: ₹{wallet}")
```

Expected Output:

```
Wallet: ₹850
```

6. For code `"INV-204"`, print whether `"INV"` is in it. Set `notes = None` and print `notes is None`.

Model answer:

```
code = "INV-204"
print("INV" in code)
notes = None
print(notes is None)
```

Expected Output:

```
True
True
```

Lab

Combine variables, cast, arithmetic, `+=` or a total, a boolean expression, and f-strings. Not the canteen GST kit.

Problem

Three friends share a restaurant bill. They want each person's share and whether the bill counts as a "large" bill (≥ ₹500).

What to Build

A script `bill_split.py`.

Requirements

- `input()` for **bill amount** (cast to `float`) and **number of friends** (cast to `int`)
- Print each share using `/` and `:.2f`
- Print whole-rupee share with `//` and leftover rupees with `%` (use `int(bill)` for those two lines, or work in whole rupees)
- Add a tip of `50` to a running `to_pay` with `+=`
- Print whether the original bill is `>= 500` (`True` / `False` — no `if`)

Expected Behavior

Share is a float with 2 decimals. The large-bill line is a boolean, not the words Pass/Fail. Invalid text still raises `ValueError` — that is expected this week.

Sample Run

```
Bill amount: 900
Friends: 3
Each pays: ₹300.00
Whole rupees each: 300
Leftover rupees: 0
To pay with tip: ₹950.00
Large bill: True
```

(Your prompts may differ; the **shape** should match.)

Constraints

No `if`, no `for`, no `while`, no `def`. One run, one bill.

Bonus

Apply 5% GST to the bill **before** splitting. Print the GST-inclusive total with 2 decimals.

Mini-Project (Assignment): Campus Canteen Calculator

Why this matters: A counter line is `price × qty + GST`. Every checkout screen does this math. Casting the typed text is the bug that crashes beginners.

Lab kit: 15_Lab_Assets/06_cli_calculator/
Also work through 15_Lab_Assets/07_casting_bugs/ — marks entry that crashes on `78.5`.

File	What you do
`06_cli_calculator/starter/`	Build **one** canteen line: price × qty + **5% GST**
`07_casting_bugs/broken/`	Fix bad casts (`int` on `78.5`, truncated average)
`solution/`	Do **not** copy this

This is **not** the marks assemble from class and **not** the bill-split lab. Same tools, canteen ticket.

Objective

Read price and qty, compute GST 5%, print a rupee total with 2 decimals.

Story / Scenario

Campus canteen staff still use paper for `price × qty + GST`. Build the CLI for one order line.

Requirements

Piece	Detail
Inputs	`float` price, `int` qty
Math	`subtotal = price * qty`; `total = subtotal * (1 + 0.05)` (round to 2 decimals)
Display	f-string like `Total: ₹105.00`
Style	Script only — **no** `if` / `while` / `def`

Expected Behavior

- `"40"` and `"2"` → total **₹84.00** (80 + 5% GST)
- `broken/calculator.py` in the kit uses `//` and adds GST as rupees — fix that pattern if you open it

Sample Run

```
Price: 40
Qty: 2
Total: ₹84.00
```

Bonus Challenges

- After Flow Control: reject non-positive price/qty with `if`; re-ask with `while`
- After Exceptions: reject non-numeric input without crashing
- Fix every broken cast in `07_casting_bugs`

Done when

- [ ] One price and one qty produce a GST 5% total
- [ ] Amount prints with ₹ and 2 decimals
- [ ] No `if` / `while` / `def` on the core path
- [ ] Casting-bugs kit: marks accept `78.5` and average shows 2 decimals

---

Advanced / Reference

Column padding (optional): `f"{'Item':<10}{'Price':>8}"` lines up a mini table. Handy later for receipts — not required for this lab.

Floats and money: `0.1 + 0.2 == 0.3` can be `False` (binary floats are approximate). Display with `:.2f`. Production money systems may use `decimal.Decimal` later — `float` + `:.2f` is enough for these CLIs.

Interview Q&A

Q1. Name Python's basic scalar types.
`int`, `float`, `str`, `bool`, and `None` (`NoneType`).

Q2. `/` vs `//`?
`/` → float division (`7 / 2 → 3.5`). `//` → floor division (`7 // 2 → 3`).

Q3. What is an f-string?
A string prefixed with `f` that embeds expressions: `f"Total: {total}"`.

Q4. Is Python strongly typed?
Yes — `"5" + 5` fails; you must cast. It is also **dynamically** typed (no type declaration required).

Q5. `is` vs `==` for `None`?
Use `x is None`. `is` checks identity; `==` checks value.

Q6. Why is an employee ID often a `str`?
Leading zeros and codes like `E1024` must not become numbers.

Q7. Why can `0.1 + 0.2 == 0.3` be False?
Binary floats are approximate. Round for display. Serious money code may use `decimal.Decimal` later.

Q8. What does `input()` return?
Always a `str`. Cast with `int` or `float` before math.

Cheat Sheet

```
name = value          → bind a labelled box
type(x)               → int / float / str / bool / NoneType
x is None             → missing check (not == None)
+ - * / // % **       → arithmetic  (/ float, // floor, % remainder)
== != < > <= >=       → comparisons → True/False
and  or  not          → combine booleans
```

```
+= -= *=              → update in place
"@" in email          → membership (this week: strings)
input("…")            → always str
int("42")  float("1.5")  str(42)
f"{n:.2f}"  f"{n:,}"  → 2 decimals / thousands
```

Python 3: `5 / 2` is `2.5`. IDs stay `str`.

Completion Checklist

```
[ ] I can store values and reuse them in later lines
[ ] I can pick int / float / str / bool / None and check with type()
[ ] I can use / // % and +=
[ ] I can print comparison and and/or results without if
[ ] I can cast input() before math
[ ] I can print ₹ amounts with f-strings
[ ] I completed the bill-split lab
[ ] I attempted the canteen calculator and casting-bugs kits
[ ] I can explain / vs // and is None in an interview
```

What's Next?

You can now remember values, compute, and read typed input.

Next, Flow Control wraps these `True` / `False` expressions in **`if`** / **`elif`** / **`else`**, and teaches **`for`** / **`while`**, so a program can choose a path and repeat — menus, PIN retries, pass/fail — instead of running in a straight line.